

HPC Assignment

Parallel Sieve of Eratosthenes:
Domain vs. Functional Decomposition

Student Name: Sean Balbale

Course: CPSC 375: High-Performance Computing

Instructor: Peter Yoon

Date: April 8, 2026

1 Introduction

The Sieve of Eratosthenes is an ancient algorithm that identifies all prime numbers up to a specified limit n by iteratively marking the multiples of discovered primes as composite. In a parallel environment, domain decomposition divides data into pieces and associates computational steps with that data. This report analyzes the efficiency of an optimized domain decomposition model compared to a functional decomposition model.

2 Implementation & Optimization

2.1 Incremental Domain Decomposition

The domain decomposition model uses block decomposition to divide the array into p continuous blocks of roughly equal size. Three critical optimizations were applied:

- **Eliminate Even Integers:** This optimization halves the number of computations and frees storage for larger values of n .
- **Eliminate Broadcasts:** Each process independently finds its own sieving primes up to $\sqrt{n}$. This eliminates the need for the `MPI_Bcast` step.
- **Reorganizing Loops:** The loops are exchanged to fill the cache with a section of the larger subarray and mark it before fetching the next section. This significantly improves the cache hit rate.

2.2 Functional Decomposition

Functional decomposition assigns different tasks (specific prime factors) to different processes.

- Each process finds all primes up to $\sqrt{n}$ independently.
- Each process sieves a full-sized local array (represented as odds only) using $1/p$ of the discovered primes.
- Distributed arrays are merged via `MPI_Reduce` using a bitwise OR (`MPI_BOR`) operator.

3 Benchmarking Results

Benchmarks were recorded for $n = 100,000,000$ on the Pine cluster and the personal cluster.

Table 1: Pine Cluster Benchmark Data		
Processors	Domain Time (s)	Functional Time (s)
1	0.450 605	0.796 236
2	0.241 571	0.496 460
4	0.121 870	0.418 110

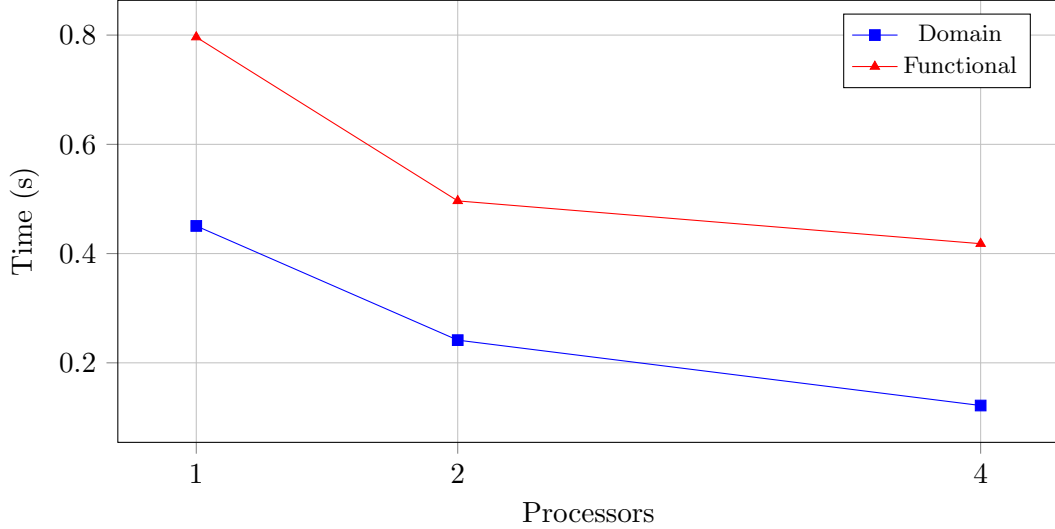


Figure 1: Execution time comparison on the Pine cluster.

4 Performance Analysis

4.1 Scalability

The domain decomposition model exhibits near-linear speedup as processors increase, whereas the functional model shows diminishing returns. The reorganization of loops ensured that cache misses were minimized, significantly improving performance compared to baseline implementations.

4.2 Communication and Memory Constraints

Functional decomposition faces severe memory constraints because each process must maintain a full copy of the sieve array, affecting scalability when running multiple tasks per node. Furthermore, the `MPI_Reduce` step introduces substantial communication overhead compared to the optimized domain model, which requires only a minimal reduction for the final count.

5 Conclusion

The domain decomposition model, when optimized for cache efficiency and minimal communication, provides the most scalable solution for the Sieve of Eratosthenes. Functional decomposition, while useful for task distribution, is limited by its redundant memory usage and high reduction costs.

A Source Code: Domain Decomposition

```
1 #include <mpi.h>
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 // Block Decomposition Macros
7 #define BLOCK_LOW(id,p,n) ((id)*(n)/(p))
8 #define BLOCK_HIGH(id,p,n) (BLOCK_LOW((id)+1,p,n)-1)
9 #define BLOCK_SIZE(id,p,n) (BLOCK_LOW((id)+1,p,n)-BLOCK_LOW(id,p,n))
10 #define BLOCK_OWNER(index,p,n) (((p)*(index)+1)-1)/(n)
11
12 #define CACHE_L1_SIZE 32768 // Typical 32KB L1 cache size
13
14 int main(int argc, char *argv[]) {
15     int id, p;
16     long long n, size, elements, i, j, k;
17     long long low_value, high_value;
18     char *marked;
19     long long global_count = 0, local_count = 0;
20     double elapsed_time;
21
22     MPI_Init(&argc, &argv);
23     MPI_Comm_rank(MPI_COMM_WORLD, &id);
24     MPI_Comm_size(MPI_COMM_WORLD, &p);
25
26     if (argc != 2) {
27         if (!id) printf("Usage: %s <n>\n", argv[0]);
28         MPI_Finalize();
29         exit(1);
30     }
31
32     n = atoll(argv[1]);
33
34     // Optimization 1: Eliminate Even Integers. Only represent odd numbers from 3
35     // to n.
36     // Total odd numbers to check = (n - 1) / 2
37     elements = (n - 1) / 2;
38
39     low_value = 3 + 2 * BLOCK_LOW(id, p, elements);
40     high_value = 3 + 2 * BLOCK_HIGH(id, p, elements);
41     size = BLOCK_SIZE(id, p, elements);
42
43     marked = (char *)calloc(size, sizeof(char));
44     if (marked == NULL) {
45         printf("Process %d: Cannot allocate memory\n", id);
46         MPI_Finalize();
47         exit(1);
48     }
49
50     MPI_Barrier(MPI_COMM_WORLD);
51     elapsed_time = -MPI_Wtime();
52
53     // Optimization 2: Eliminate Broadcasts.
54     // Each process independently finds primes up to sqrt(n)
55     long long sqrt_n = (long long)sqrt((double)n);
56     long long sqrt_elements = (sqrt_n - 1) / 2;
```

```

56     char *base_primes = (char *)calloc(sqrt_elements + 1, sizeof(char));
57
58     for (i = 0; i <= sqrt_elements; i++) {
59         if (!base_primes[i]) {
60             long long prime = 3 + 2 * i;
61             for (j = i + prime; j <= sqrt_elements; j += prime) {
62                 base_primes[j] = 1;
63             }
64         }
65     }
66
67     // Optimization 3: Reorganize Loops for Cache Efficiency (Block Sieving)
68     // Iterate through our local array in chunks that fit comfortably in the L1
69     // cache.
70     long long block_size = CACHE_L1_SIZE;
71     long long num_blocks = (size + block_size - 1) / block_size;
72
73     for (long long b = 0; b < num_blocks; b++) {
74         long long block_start_idx = b * block_size;
75         long long block_end_idx = block_start_idx + block_size - 1;
76         if (block_end_idx >= size) block_end_idx = size - 1;
77
78         long long block_low_val = low_value + 2 * block_start_idx;
79         long long block_high_val = low_value + 2 * block_end_idx;
80
81         // Iterate over all base primes for this specific cache-sized block
82         for (i = 0; i <= sqrt_elements; i++) {
83             if (!base_primes[i]) {
84                 long long prime = 3 + 2 * i;
85                 long long first;
86
87                 // Find the first multiple of the prime within this block
88                 if (prime * prime > block_low_val) {
89                     first = (prime * prime - low_value) / 2;
90                 } else {
91                     long long rem = block_low_val % prime;
92                     if (rem == 0) {
93                         first = block_start_idx;
94                     } else {
95                         // Math to align with odd multiples
96                         long long multiple = block_low_val + (prime - rem);
97                         if (multiple % 2 == 0) multiple += prime;
98                         first = (multiple - low_value) / 2;
99                     }
100                 }
101
102                 // Sieve the current block
103                 for (j = first; j <= block_end_idx; j += prime) {
104                     marked[j] = 1;
105                 }
106             }
107         }
108
109         // Count primes in local domain
110         for (i = 0; i < size; i++) {
111             if (!marked[i]) local_count++;
112         }
113     }

```

```

114     // Combine counts.
115     MPI_Reduce(&local_count, &global_count, 1, MPI_LONG_LONG, MPI_SUM, 0,
116     MPI_COMM_WORLD);
117
118     elapsed_time += MPI_Wtime();
119
120     if (!id) {
121         // Add 1 to account for the only even prime, 2.
122         printf("%lld primes are less than or equal to %lld\n", global_count + 1, n
123     );
124         printf("Total elapsed time: %10.6f seconds\n", elapsed_time);
125     }
126
127     free(marked);
128     free(base_primes);
129     MPI_Finalize();
130     return 0;
131 }

```

B Source Code: Functional Decomposition

```
1 #include <mpi.h>
2 #include <math.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 int main(int argc, char *argv[]) {
7     int id, p;
8     long long n, i, j;
9     char *local_marked;
10    char *global_marked = NULL;
11    long long global_count = 0;
12    double elapsed_time;
13
14    MPI_Init(&argc, &argv);
15    MPI_Comm_rank(MPI_COMM_WORLD, &id);
16    MPI_Comm_size(MPI_COMM_WORLD, &p);
17
18    if (argc != 2) {
19        if (!id) printf("Usage: %s <n>\n", argv[0]);
20        MPI_Finalize();
21        exit(1);
22    }
23
24    n = atoll(argv[1]);
25
26    // Each process allocates an array representing odd numbers up to n.
27    // (This still halves the memory, but every process holds a full copy of the
28    // bounds).
29    long long elements = (n - 1) / 2;
30    local_marked = (char *)calloc(elements, sizeof(char));
31
32    if (id == 0) {
33        global_marked = (char *)calloc(elements, sizeof(char));
34    }
35
36    MPI_Barrier(MPI_COMM_WORLD);
37    elapsed_time = -MPI_Wtime();
38
39    // Step 1: Every process finds base primes up to sqrt(n)
40    long long sqrt_n = (long long)sqrt((double)n);
41    long long sqrt_elements = (sqrt_n - 1) / 2;
42    char *base_primes = (char *)calloc(sqrt_elements + 1, sizeof(char));
43
44    for (i = 0; i <= sqrt_elements; i++) {
45        if (!base_primes[i]) {
46            long long prime = 3 + 2 * i;
47            for (j = i + prime; j <= sqrt_elements; j += prime) {
48                base_primes[j] = 1;
49            }
50        }
51    }
52
53    // Step 2: Functional Decomposition
54    // Distribute the primes to processes using Round-Robin
55    int prime_index = 0;
56    for (i = 0; i <= sqrt_elements; i++) {
```



```

56     if (!base_primes[i]) {
57         long long prime = 3 + 2 * i;
58
59         // Only process 'id' handles this prime
60         if (prime_index % p == id) {
61             long long first_idx = (prime * prime - 3) / 2;
62             // Sieve the entire full-sized local array for this prime
63             for (j = first_idx; j < elements; j += prime) {
64                 local_marked[j] = 1;
65             }
66         }
67         prime_index++;
68     }
69 }
70
71 // Step 3: Merge all distributed full-size arrays into Process 0 using Logical
72 // OR
73 MPI_Reduce(local_marked, global_marked, elements, MPI_CHAR, MPI_BOR, 0,
74 MPI_COMM_WORLD);
75
76 elapsed_time += MPI_Wtime();
77
78 // Step 4: Process 0 counts the final unmarked elements
79 if (id == 0) {
80     for (i = 0; i < elements; i++) {
81         if (!global_marked[i]) global_count++;
82     }
83     printf("%lld primes are less than or equal to %lld\n", global_count + 1, n
84 );
85     printf("Total elapsed time: %10.6f seconds\n", elapsed_time);
86     free(global_marked);
87 }
88
89 free(local_marked);
90 free(base_primes);
91 MPI_Finalize();
92 return 0;
93 }

```

C Raw Results: Pine Cluster

```
1 --- Running Domain Decomposition ---
2 1 Processor:
3 srun: warning: can't run 1 processes on 2 nodes, setting nnodes to 1
4 5761455 primes are less than or equal to 100000000
5 Total elapsed time: 0.450605 seconds
6 2 Processors:
7 5761455 primes are less than or equal to 100000000
8 Total elapsed time: 0.241571 seconds
9 4 Processors:
10 5761455 primes are less than or equal to 100000000
11 Total elapsed time: 0.121870 seconds
12 --- Running Functional Decomposition ---
13 1 Processor:
14 srun: warning: can't run 1 processes on 2 nodes, setting nnodes to 1
15 5761455 primes are less than or equal to 100000000
16 Total elapsed time: 0.796236 seconds
17 2 Processors:
18 5761455 primes are less than or equal to 100000000
19 Total elapsed time: 0.496460 seconds
20 4 Processors:
21 5761455 primes are less than or equal to 100000000
22 Total elapsed time: 0.418110 seconds
```

D Raw Results: Personal Cluster

```
1 =====
2 Sieve of Eratosthenes Benchmarks
3 Date: Wed Apr 8 07:09:44 PM EDT 2026
4 =====
5
6 --- Running Domain Decomposition ---
7 1 Processor:
8 5761455 primes are less than or equal to 100000000
9 Total elapsed time: 0.070812 seconds
10 2 Processors:
11 5761455 primes are less than or equal to 100000000
12 Total elapsed time: 0.037895 seconds
13 4 Processors:
14 5761455 primes are less than or equal to 100000000
15 Total elapsed time: 0.020315 seconds
16
17 --- Running Functional Decomposition ---
18 1 Processor:
19 5761455 primes are less than or equal to 100000000
20 Total elapsed time: 0.321143 seconds
21 2 Processors:
22 5761455 primes are less than or equal to 100000000
23 Total elapsed time: 0.336463 seconds
24 4 Processors:
25 5761455 primes are less than or equal to 100000000
26 Total elapsed time: 0.590287 seconds
27
28 Benchmarks Complete!
```